DynamicServiceOrchestration

System Design Description

Abstract
This document describes the design and implementation of the DynamicServiceOrchestration core system of Arrowhead (documentation edition 5.3.0) in Java Spring (`ah5-core-java-spring/serviceorchestration-dynamic`).

Contents

1 Documentation edition and implementation scope **3**

2 Overview **3**

3 Implementation **4**
3.1 Implementation language and tools 4
3.2 Functional properties implementation 4
3.3 Non functional properties implementation 5
3.4 Deployment 5
3.5 Local modifications 6
3.6 Verification 6

4 Services **7**

5 Go and Rust implementation variants **7**
5.1 Go implementation 7
5.2 Rust implementation 8

6 References **8**

7 Revision History **11**
7.1 Amendments 11
7.2 Quality Assurance 11

1 Documentation edition and implementation scope

This is the 5.3.0 documentation edition, dated 2026-11-09. It carries forward the system and service contracts from the retained 5.2.2 documentation and identifies the current primary implementation sources. The former `documentation/v5.2.2` folder is retained unchanged.

Document version 5.3.0 is distinct from executable, dependency, registered system/service and wire-contract versions. Values still defined by the source (including 5.2.2 artefact versions, 1.0.0 service versions and `SysMonitor-AMQP-IDD-5.2.2` payload identifiers) are preserved. This documentation update does not change runtime behaviour or establish a new implementation release. Historical verification statements describe the tests recorded in the inherited baseline; this revision validates document compilation and references.

2 Overview

This document describes the DynamicServiceOrchestration system implementation, which realizes the DynamicServiceOrchestration SysD [1]. DynamicServiceOrchestration finds, at request time, the providers a consumer shall use. It discovers candidates in the ServiceRegistry, keeps only the authorized, not blacklisted and not locked ones with matching interfaces, applies preferences, QoS requirements, matchmaking and exclusivity, obtains the authorization tokens the providers require, and returns the result; it also pushes results to subscribers. In this workspace it extends orchestration to neighbour clouds through the Gatekeeper, and enforces the active-plant consumer/service scopes submitted by PlantDescriptionEngine. In Section 3, we describe implementation details of the system.

3 Implementation

This implementation is based on the SysD document DynamicServiceOrchestration [1], the upstream Eclipse Arrowhead Java Spring implementation 5.2.2 [2] and the local changes listed in Section 3.5.

3.1 Implementation language and tools

- Java 21, Spring Boot 3.4.9 (web, security, data JPA, Quartz), springdoc-openapi 2.8 (Swagger UI), Log4j 2.
- Maven module `serviceorchestration-dynamic` of `ah5-core-java-spring`; it depends on `arrowhead-common-utils` and `arrowhead-data-transfer-objects` 5.2.2 from the local `ah5-common-java-spring` (Eclipse Paho MQTT v3 1.2.5, RabbitMQ AMQP client 5.22.0).
- Database: MariaDB/MySQL (MariaDB Connector/J; MySQL 8.4 server in the test cloud), schema validated by Hibernate (`ddl-auto=validate`); SQL scripts in `src/main/resources/database`.
- Stateless request processing; all state is in the database. Scheduled jobs use Quartz.

3.2 Functional properties implementation

The Java packages below `eu.arrowhead` implement the system:

Package	Responsibility
`api.http, api.mqtt`	Controllers and topic handlers of the five services.
`service.utils.LocalService Orchestration`	The local orchestration pipeline.
`service.utils.InterCloud ServiceOrchestration`	Inter-cloud orchestration through the Gatekeeper (local extension).
`service.utils.matchmaker`	Selection of one candidate (`DefaultServiceInstance Matchmaker`).
`driver.QoSDriver`	QoS evaluation through DeviceQoSEvaluator.
`service.thread`	Push orchestration thread and workers.
`service, service.model, service.validation`	Orchestration, push, lock and history logic; request validation and normalization.
`jpa, quartz`	Job, subscription, lock, QoS result and log entities; cleaner job.

Every request passes the common filter chain: authentication filter (by `authentication.policy`), optional blacklist filter, management service filter (for management services), then the controller, validation and normalization, the service logic and the database service. The same logic is reachable through the MQTT topic handlers.

3.2.1 Data handled and stored

Database `ah_serviceorchestration_dynamic`:

Tables	Content
`orchestration_job`	Pull and push jobs with status and message.
`subscription`	Push subscriptions with request and notify interface.
`orchestration_lock`	Exclusive and temporary locks.
`qos_eval_result`	Results of QoS evaluations.
`logs`	Log entries for `get-log`.

The data models of the interfaces are the Java records of `eu.arrowhead.dto` (common library); the IDD documents name them without the `DTO` suffix. The records used by this system are: `AuthorizationTokenGenerationResponseDTO`, `KeyValuesDTO`, `LogEntryDTO`, `LogEntryListResponseDTO`, `LogRequestDTO`, `MetadataRequirementDTO`, `OrchestrationHistoryQueryRequestDTO`, `OrchestrationHistoryResponseDTO`, `OrchestrationJobDTO`, `OrchestrationLockListRequestDTO`, `OrchestrationLockListResponseDTO`, `OrchestrationLockQueryRequestDTO`, `OrchestrationLockRequestDTO`, `OrchestrationLockResponseDTO`, `OrchestrationNotifyInterfaceDTO`, `OrchestrationPlantProviderDTO`, `OrchestrationPlantRuleDTO`, `OrchestrationPlantRulesDTO`, `OrchestrationPushJobListResponseDTO`, `OrchestrationPushTriggerDTO`, `OrchestrationRequestDTO`, `OrchestrationResponseDTO`, `OrchestrationResultDTO`, `OrchestrationServiceRequirementDTO`, `OrchestrationSubscriptionListRequestDTO`, `OrchestrationSubscriptionListResponseDTO`, `OrchestrationSubscriptionQueryRequestDTO`, `OrchestrationSubscriptionRequestDTO`, `OrchestrationSubscriptionResponseDTO`, `PageDTO`, `QoSRequirementDTO`, `ServiceInstanceInterfaceResponseDTO`.

3.3 Non functional properties implementation

3.3.1 Security

TLS and client authentication are configured with the Spring `server.ssl.*` properties (PKCS#12 keystore and truststore). The authentication filters of the common library implement the four authentication policies; `TokenCertificateBinding` binds identity tokens to the client certificate and `CrlRevocationConfig` enables CRL checking in Tomcat (both local). The private key of the keystore is also used to sign the system's outgoing requests where required.

3.3.2 Power management

None; the system is a server process without power states. The `sysMonitor` `power` observation always reports line supply.

3.3.3 Internal monitoring

Log4j 2 logs to stdout and rolling files; log entries are also written to the `logs` table and served by `generalManagement.get-log`. The native `sysMonitor` publisher (`eu.arrowhead.common.sysmonitor`) publishes `echo`, `power` and `status` every 30 s.

3.3.4 Configuration

Configuration is read from `application.properties` and environment variables (Spring relaxed binding, e.g. `SERVICE_DISCOVERY_POLICY`); secrets come from files through `SPRING_CONFIG_IMPORT=configtree:/run/secrets/`. The properties are listed in the SysD. `generalManagement.get-config` returns the public ones.

3.4 Deployment

The test cloud (`Arrowhead-test-cloud-java`) builds the image `arrowhead-serviceorchestration-dynamic:5.2.2-sysmonitor` from the released image `aitiaiiot/arrowhead-serviceorchestration-dynamic:5.2.2` with the locally built `arrowhead-serviceorchestration-dynamic-5.2.2.jar` (`java-patch/Dockerfile`). The container runs read-only with the configuration baked into `/app/config`, only its keystores mounted, and publishes port 8441 on the loopback address of the host. Its database runs in a separate container.

3.5 Local modifications

Compared to upstream 5.2.2, this workspace changes:

- Startup is refused when the effective configuration is below the secure profile (`SecurityProfileGuard`).
- Inter-cloud orchestration through the Gatekeeper; `ALLOW_INTERCLOUD` falls back to it without local results, `ONLY_INTERCLOUD` goes straight to it. Warnings `interCloud`, `interCloudDisabled`, `interCloudFailed`.
- Job messages name the selected providers (`<n> local result: <providers>`, `<n> intercloud result: <provider>@<cloud>`).
- The HTTP `monitor` service (`/serviceorchestration/monitor/echo`) was removed; telemetry is the native `sysMonitor` service.
- `http.collector.bootstrap.services` also lists the authorization and blacklist services, which the orchestrator cannot orchestrate for itself.
- Token-to-certificate binding, CRL checking, MariaDB driver.

3.6 Verification

The module has 30 JUnit test classes; the regression report set recorded on 2026-11-07 covers 477 tests without failures. Live-cloud smoke tests are a separate verification step; the plant-management migration is not claimed as deployed by this update.

4 Services

The implementation of the services is based on the SD and IDD documents referenced below.

Services produced	**SysD ref**	**SD ref**	**IDD ref**
`serviceOrchestration`	[1]	[3]	[4], [5]
`serviceOrchestrationPush Management`	[1]	[6]	[7], [8]
`serviceOrchestrationLock Management`	[1]	[9]	[10], [11]
`serviceOrchestrationHistory Management`	[1]	[12]	[13], [14]
`serviceOrchestrationPlant Management`	[1]	[15]	[16]
`generalManagement`	[1]	[17]	[18], [19]
`sysMonitor`	[1]	[20]	[21]
Services consumed	**SysD ref**	**SD ref**	**IDD ref**
serviceDiscovery	[22]	[23]	–
identity	[24]	[25]	–
authorization; authorization Management	[26]	[27]	–
authorizationTokenManagement	[26]	[28]	–
blacklistDiscovery; blacklist Management	Blacklist	–	–
qualityEvaluation	DeviceQo SEvaluator	–	–
translationBridgeManagement	Translation Manager	–	–
globalServiceDiscovery; interCloud Negotiation	Gatekeeper	–	–
notify interface	Subscribers	–	–
sysMonitor (publisher)	AMQP broker (SysMonitor)	–	–

5 Go and Rust implementation variants

5.1 Go implementation

Primary system folder: `Eclipse-Arrowhead-go/ah5-core-go/serviceorchestration-dynamic`.

The system owns its Go module, launcher and build configuration. Use the per-system `Makefile` and `go.mod`; the containing Go tree is the build source, not a historical Arrowhead copy. Ordinary native roles use independent identities and JSON snapshot state rather than the Java database. Do not assume Java SQL migration, clustering or identical transaction/retry semantics. The registry has its own configuration and optional native AMQP monitor; other roles must be checked against their own registrations.

Listening port. Built-in default 127.0.0.1:8441 (Arrowhead default). The shipped config.json sets "listen": "0.0.0.0:8441". Override with "listen" in the configuration file, SERVER_PORT=N or -server.port N; the port overrides keep the configured host. Native design/route evidence: `Eclipse-Arrowhead-go/ah5-core-go/serviceorchestration-dynamic/documentation`.

For this Java interface, select only a native transport and operation explicitly implemented and advertised by that port. A matching method/path does not establish DTO, error, authorization, retry or persistence equivalence. Java MQTT/AMQP bindings are not automatically provided by a Go or Rust role. Differential contract and security checks remain necessary before substitution.

5.2 Rust implementation

Primary system folder: `Eclipse-Arrowhead-rust/ah5-core-rust/serviceorchestration-dynamic`.

The per-system entry point is built by the `arrowhead-core-rust` crate. Build from the primary Rust workspace with the applicable package/binary selection. Documentation links can resolve into the shared core/support catalogue. Independent process identity and state remain required.

The ordinary native profile uses Tokio/Axum and rustls with peer-identity checks and independent locked JSON state. Network effects are not distributed database transactions. The precise trust, CRL reload and persistence guarantees are those of the native profile, not the Java implementation.

Native design/route evidence: `Eclipse-Arrowhead-rust/ah5-core-rust/documentation/DynamicServiceOrchestration`.

For this Java interface, select only a native transport and operation explicitly implemented and advertised by that port. A matching method/path does not establish DTO, error, authorization, retry or persistence equivalence. Java MQTT/AMQP bindings are not automatically provided by a Go or Rust role. Differential contract and security checks remain necessary before substitution.

6 References

[1] J. Delsing, “DynamicServiceOrchestration – System Description,” Eclipse Arrowhead, SysD 5.3.0, 2026, serviceorchestration-dynamic/documentation/v5.3.0/DynamicServiceOrchestration-v5.3.0-SysD.pdf.

[2] AITIA International, “Eclipse arrowhead 5 core systems (java spring) and documentation,” https://aitia-iiot.github.io/ah5-docs-java-spring/, 2025.

[3] J. Delsing, “serviceOrchestration – Service Description (DynamicServiceOrchestration),” Eclipse Arrowhead, SD 5.3.0, 2026, dynamicServiceOrchestration-v5.3.0-serviceOrchestration-SD.pdf.

[4] ——, “serviceOrchestration HTTP/TLS/JSON – Interface Design Description (DynamicServiceOrchestration),” Eclipse Arrowhead, IDD 5.3.0, 2026, dynamicServiceOrchestration-v5.3.0-serviceOrchestration-HTTP-IDD.pdf.

[5] ——, “serviceOrchestration MQTT/TLS/JSON – Interface Design Description (DynamicServiceOrchestration),” Eclipse Arrowhead, IDD 5.3.0, 2026, dynamicServiceOrchestration-v5.3.0-serviceOrchestration-MQTT-IDD.pdf.

[6] ——, “serviceOrchestrationPushManagement – Service Description (DynamicServiceOrchestration),” Eclipse Arrowhead, SD 5.3.0, 2026, dynamicServiceOrchestration-v5.3.0-serviceOrchestrationPushManagement-SD.pdf.

[7] ——, “serviceOrchestrationPushManagement HTTP/TLS/JSON – Interface Design Description (DynamicServiceOrchestration),” Eclipse Arrowhead, IDD 5.3.0, 2026, dynamicServiceOrchestration-v5.3.0-serviceOrchestrationPushManagement-HTTP-IDD.pdf.

[8] ——, “serviceOrchestrationPushManagement MQTT/TLS/JSON – Interface Design Description (DynamicServiceOrchestration),” Eclipse Arrowhead, IDD 5.3.0, 2026, dynamicServiceOrchestration-v5.3.0-serviceOrchestrationPushManagement-MQTT-IDD.pdf.

[9] ——, “serviceOrchestrationLockManagement – Service Description (DynamicServiceOrchestration),” Eclipse Arrowhead, SD 5.3.0, 2026, dynamicServiceOrchestration-v5.3.0-serviceOrchestrationLockManagement-SD.pdf.

[10] ——, “serviceOrchestrationLockManagement HTTP/TLS/JSON – Interface Design Description (DynamicServiceOrchestration),” Eclipse Arrowhead, IDD 5.3.0, 2026, dynamicServiceOrchestration-v5.3.0-serviceOrchestrationLockManagement-HTTP-IDD.pdf.

[11] ——, “serviceOrchestrationLockManagement MQTT/TLS/JSON – Interface Design Description (DynamicServiceOrchestration),” Eclipse Arrowhead, IDD 5.3.0, 2026, dynamicServiceOrchestration-v5.3.0-serviceOrchestrationLockManagement-MQTT-IDD.pdf.

[12] ——, “serviceOrchestrationHistoryManagement – Service Description (DynamicServiceOrchestration),” Eclipse Arrowhead, SD 5.3.0, 2026, dynamicServiceOrchestration-v5.3.0-serviceOrchestrationHistoryManagement-SD.pdf.

[13] ——, “serviceOrchestrationHistoryManagement HTTP/TLS/JSON – Interface Design Description (DynamicServiceOrchestration),” Eclipse Arrowhead, IDD 5.3.0, 2026, dynamicServiceOrchestration-v5.3.0-serviceOrchestrationHistoryManagement-HTTP-IDD.pdf.

[14] ——, “serviceOrchestrationHistoryManagement MQTT/TLS/JSON – Interface Design Description (DynamicServiceOrchestration),” Eclipse Arrowhead, IDD 5.3.0, 2026, dynamicServiceOrchestration-v5.3.0-serviceOrchestrationHistoryManagement-MQTT-IDD.pdf.

[15] ——, “serviceOrchestrationPlantManagement – Service Description (DynamicServiceOrchestration),” Eclipse Arrowhead, SD 5.3.0, 2026, dynamicServiceOrchestration-v5.3.0-serviceOrchestrationPlantManagement-SD.pdf.

[16] ——, “serviceOrchestrationPlantManagement HTTP/TLS/JSON – Interface Design Description (DynamicServiceOrchestration),” Eclipse Arrowhead, IDD 5.3.0, 2026, dynamicServiceOrchestration-v5.3.0-serviceOrchestrationPlantManagement-HTTP-IDD.pdf.

[17] ——, “generalManagement – Service Description (DynamicServiceOrchestration),” Eclipse Arrowhead, SD 5.3.0, 2026, dynamicServiceOrchestration-v5.3.0-generalManagement-SD.pdf.

[18] ——, “generalManagement HTTP/TLS/JSON – Interface Design Description (DynamicServiceOrchestration),” Eclipse Arrowhead, IDD 5.3.0, 2026, dynamicServiceOrchestration-v5.3.0-generalManagement-HTTP-IDD.pdf.

[19] ——, “generalManagement MQTT/TLS/JSON – Interface Design Description (DynamicServiceOrchestration),” Eclipse Arrowhead, IDD 5.3.0, 2026, dynamicServiceOrchestration-v5.3.0-generalManagement-MQTT-IDD.pdf.

[20] ——, “sysMonitor – Service Description (DynamicServiceOrchestration),” Eclipse Arrowhead, SD 5.3.0, 2026, dynamicServiceOrchestration-v5.3.0-sysMonitor-SD.pdf.

[21] ——, “sysMonitor AMQP/TLS/JSON – Interface Design Description (DynamicServiceOrchestration),” Eclipse Arrowhead, IDD 5.3.0, 2026, dynamicServiceOrchestration-v5.3.0-sysMonitor-AMQP-IDD.pdf.

[22] ——, “ServiceRegistry – System Description,” Eclipse Arrowhead, SysD 5.3.0, 2026, serviceregistry/documentation/v5.3.0/ServiceRegistry-v5.3.0-SysD.pdf.

[23] ——, “serviceDiscovery – Service Description (ServiceRegistry),” Eclipse Arrowhead, SD 5.3.0, 2026, serviceRegistry-v5.3.0-serviceDiscovery-SD.pdf.

[24] ——, “Authentication – System Description,” Eclipse Arrowhead, SysD 5.3.0, 2026, authentication/documentation/v5.3.0/Authentication-v5.3.0-SysD.pdf.

[25] ——, “identity – Service Description (Authentication),” Eclipse Arrowhead, SD 5.3.0, 2026, authentication-v5.3.0-identity-SD.pdf.

[26] ——, “ConsumerAuthorization – System Description,” Eclipse Arrowhead, SysD 5.3.0, 2026, consumerauthorization/documentation/v5.3.0/ConsumerAuthorization-v5.3.0-SysD.pdf.

[27] ——, “authorization – Service Description (ConsumerAuthorization),” Eclipse Arrowhead, SD 5.3.0, 2026, consumerAuthorization-v5.3.0-authorization-SD.pdf.

[28] ——, “authorizationTokenManagement – Service Description (ConsumerAuthorization),” Eclipse Arrowhead, SD 5.3.0, 2026, consumerAuthorization-v5.3.0-authorizationTokenManagement-SD.pdf.

7 Revision History

7.1 Amendments

No.	Date	Version	Subject of Amendments	Author
1	2026-11-07	5.2.2	Document created from the Arrowhead LaTeX templates for the 5.2.2 Java Spring implementation, including the local extensions of this workspace.	Jerker Delsing
2	2026-11-09	5.3.0	New documentation edition; primary-source references and Go/Rust implementation scope; 5.2.2 retained.	Documentation maintenance

7.2 Quality Assurance

No.	Date	Version	Approved by
1		5.3.0	